

Cryptography in machine learning

The intersection of cryptography and artificial intelligence, often called Privacy-Preserving Machine Learning (PPML), offers some of the most complex engineering challenges today.

Executive Summary: This document outlines an advanced engineering portfolio at the intersection of cryptography and artificial intelligence, commonly referred to as Privacy-Preserving Machine Learning (PPML). The portfolio addresses complex engineering challenges where traditional machine learning models and sensitive datasets collide with strict regulatory constraints, data privacy demands, and security vulnerabilities. It spans six comprehensive architectural paradigms—ranging from Homomorphic Encryption and Federated Learning to Zero-Knowledge Machine Learning (zkML) and Trusted Execution Environments (TEEs)—providing production-grade Python implementations, trade-off analyses, and mitigation strategies for enterprise scaling.

Portfolio at a Glance

- * **Homomorphic Encryption (FHE):** Executes direct inference on ciphertexts via the CKKS scheme using TenSEAL.
- * **Federated Learning (FL):** Combines Differential Privacy Stochastic Gradient Descent (DP-SGD) and Federated Averaging (FedAvg) using PyTorch.
- * **Adversarial Defence & Explainability:** Integrates Isolation Forest sanitization loops with SHAP feature attribution audits.
- * **Zero-Knowledge Machine Learning (zkML):** Translates PyTorch models to ONNX and generates cryptographic SNARK proofs via EZKL.
- * **Secure Multi-Party Computation (SMPC):** Implements 3-party additive secret sharing with Beaver Triples for joint analytics.
- * **Confidential Computing (TEEs):** Leverages hardware enclaves (Intel SGX/Gramine) with remote attestation quotes for secure LLM serving.

Technical Skills

- * **Frameworks & Libraries:** TenSEAL (Microsoft SEAL), PyTorch, EZKL, Scikit-Learn, SHAP, Cryptography (Python HAZMAT), NumPy.
- * **Cryptographic & Privacy Paradigms:** Fully Homomorphic Encryption (CKKS), Differential Privacy (DP-SGD), Secret Sharing (Additive Secret Sharing, Beaver Triples), zk-SNARKs (EZKL, Halo2), TEE Remote Attestation (Intel SGX, Gramine).
- * **Engineering Patterns:** Gradient clipping, noise calibration, ONNX model exportation, fixed-point quantization, isolation forest anomaly detection, cryptographic key encapsulation, and secure gRPC/REST transport framing.

Portfolio Summary: The portfolio demonstrates how to protect machine learning systems across their entire lifecycle—securing data in transit, data in use, model parameters, and training pipelines. By replacing implicit trust in central servers with cryptographic guarantees and hardware barriers, the architecture prevents data exposure, reverses data poisoning attempts, and verifies algorithmic execution cryptographically.

Technical Philosophy

- * **Defence -in-Depth:** Security cannot rely on a single layer; robust PPML requires combining pre-input sanitization, encrypted computation, differential privacy noise, and post-hoc explainability audits.
- * **Pragmatic Performance Engineering:** Cryptographic transformations introduce performance overheads. Engineering decisions must carefully balance security guarantees (such as noise budgets in FHE or network round-trips in SMPC) against computational and latency realities.
- * **Trustlessness over Trust:** Wherever possible, cryptographic verification (zkML) or hardware enclaves (TEEs) should replace administrative trust boundaries to ensure compliance and auditability in multi-

tenant cloud environments.

Portfolio Highlights

- * **CKKS Vector Multiplication:** Successfully models separation between a Data Owner Agent and an Inference Agent, computing dot products directly on encrypted floating-point values.
- * **Differential Privacy Training Loop:** Implements client-side gradient clipping and Gaussian noise injection to prevent data leakage during decentralized SGD optimization.
- * **zkML Proof Generation Cycle:** Automates the end-to-end translation of a PyTorch network to a compiled arithmetic circuit, generating witness traces and cryptographically verifying execution without revealing private inputs.
- * **TEE Remote Attestation Handshake:** Simulates hardware-rooted security where a client verifies an enclave's code measurement (MRENCLAVE) before transmitting AES-encrypted payloads decrypted exclusively inside CPU registers.

Technical Competences Demonstrated

- * **Ciphertext Arithmetic:** Managing algebraic structure and polynomial modulus degrees for approximate number formatting.
- * **Finite-Ring Calculations:** Implementing global ring moduli for secret sharing and multi-party coordination.
- * **Circuit Compilation & Quantization:** Handling scale parameters, lookup tables, and non-linear activation approximations (e.g., ReLU mapping via Plookup).
- * **Sanitization & Anomaly Filtering:** Catching adversarial out-of-distribution inputs prior to model evaluation.

Professional Skills Demonstrated

- * **System Architecture Design:** Structuring distributed nodes, federated rounds, and zero-knowledge prover-verifier boundaries.
- * **Trade-Off Analysis:** Quantifying memory footprints, payload expansions, and compute latencies across diverse cryptographic modes.
- * **Production Readiness Planning:** Identifying enterprise integration requirements such as differential privacy budget accounting (Rényi DP), TLS wrapping, and Gramine manifest configurations.

Project 1: Homomorphic Encryption for Multi-Agent Orchestration

Fully Homomorphic Encryption (FHE) allows systems to perform mathematical computations directly on encrypted ciphertexts without ever needing a decryption key.

- * Design a multi-agent orchestration system where various agents route, process, and evaluate predictive models on encrypted data streams.
- * Execute inference tasks entirely in the ciphertext domain to ensure the central processing server never exposes the raw inputs.
- * Analyze the substantial processing and memory overhead required to maintain algebraic structure during computation.

Building a multi-agent orchestration system with Fully Homomorphic Encryption (FHE) shifts the trust paradigm. Instead of trusting a central server with raw data, the server acts purely as a mathematical engine, computing directly on encrypted vectors.

To implement this for machine learning, we typically rely on the CKKS (Cheon-Kim-Kim-Song) scheme. Unlike older FHE schemes that only handle integers, CKKS allows for approximate arithmetic over floating-point numbers, making it the standard for neural networks and regression models.

The Mathematical Foundation: In a standard linear layer (like a logistic regression or the final layer of a neural network), the prediction y is a dot product of weights W and inputs x , plus a bias b : $y = Wx + b$. Under FHE, the client encrypts x to create $Enc(x)$. The orchestration agent, holding the plain-text model weights, performs homomorphic multiplication ($\odot^*$) and addition ($\odot^+$): $Enc(y) = Enc(x) \odot^* W \odot^+ b$. The result is an encrypted prediction, $Enc(y)$, which only the client can decrypt.

Python Implementation using TenSEAL: Below is a foundational pipeline using `tenseal`, a Python library built on Microsoft SEAL, specifically designed for homomorphic encryption in machine learning. It models two distinct agents:

- * a Data Owner Agent (which handles keys and encryption) and
- * an Inference Agent (which holds the model and routes computation).

```
import tenseal as ts
import numpy as np # 1. Data Owner Agent (Client)
class DataOwnerAgent:
    def __init__(self): # Setup the FHE context using the CKKS scheme
        # poly_modulus_degree impacts security and computation limits
        context = ts.context(ts.SCHEME_TYPE.CKKS, poly_modulus_degree=8192,
coeff_mod_bit_sizes=[60, 40, 40, 60])
        context.generate_galois_keys()
        context.global_scale = 240
        self.context = context
    def encrypt_data(self, data_vector):
        """Encrypts raw floating-point features into a ciphertext."""
        return ts.ckks_vector(self.context, data_vector)
    def decrypt_result(self, encrypted_result):
        """Decrypts the prediction returned by the server."""
        return encrypted_result.decrypt() # 2. Orchestration / Inference Agent (Server)
class InferenceAgent:
    def __init__(self, weights, bias): # The server holds the trained model in plaintext
        self.weights = weights
```

```

self.bias = bias
def evaluate(self, encrypted_vector):
    """Executes the dot product and addition directly on the ciphertext. The agent never sees the raw
input or the final prediction.""" # Homomorphic dot product (ciphertext * plaintext)
    encrypted_dot_product = encrypted_vector.dot(self.weights) # Homomorphic addition (ciphertext +
plaintext)
    encrypted_prediction = encrypted_dot_product + self.bias
    return encrypted_prediction # --- Multi-Agent Execution Pipeline ---
# Initialize the model weights (e.g., from a previously trained classifier)
model_weights = [0.5, -0.2, 0.1, 0.8]
model_bias = [0.15] # Instantiate the agents
client_agent = DataOwnerAgent()
compute_agent = InferenceAgent(model_weights, model_bias) # 1. Client prepares and encrypts data
raw_input = [1.2, 3.4, 0.5, 2.1]
encrypted_input = client_agent.encrypt_data(raw_input) # 2. Transmission (simulated): Client sends
ONLY the ciphertext to the compute agent
# 3. Compute Agent executes inference on the encrypted stream
encrypted_prediction = compute_agent.evaluate(encrypted_input) # 4. Transmission (simulated):
Compute agent returns encrypted prediction
# 5. Client decrypts the final result
final_prediction = client_agent.decrypt_result(encrypted_prediction)
print(f"Homomorphic Prediction: {final_prediction[0]:.4f}")
print(f"Plaintext (Baseline): {np.dot(raw_input, model_weights) + model_bias[0]:.4f}")

```

Analyzing the FHE Overhead: Transitioning from plaintext ML to FHE introduces massive computational friction. If you deploy this in a production environment, you must account for the following trade-offs:

Resource Metric	Plaintext Inference	FHE (CKKS) Inference	Impact Analysis
Data Size (Payload)	Kilobytes	Megabytes	Ciphertext expansion increases network payload by 10x–100x, creating severe bottlenecks for REST APIs.
Compute Time	Microseconds	Milliseconds/Seconds	Matrix multiplication requires complex polynomial arithmetic, heavily taxing CPU limits.
Non-Linear Functions	Supported natively	Requires approximation	ReLU and Sigmoid cannot be directly evaluated; they must be approximated using Taylor polynomials.
Depth Limit	Infinite	Bound by Noise Budget	Every homomorphic operation adds mathematical noise. If the circuit is too deep, the data becomes unrecoverable without computationally expensive "bootstrapping."

Project 2: Federated Learning with Differential Privacy

Instead of centralizing sensitive datasets, you can build a decentralized, cryptographic training architecture.

- * Distribute the model training process across multiple local nodes that compute weight updates locally.
- * Implement Federated Averaging to aggregate the model weights securely at a central server.
- * Inject calibrated mathematical noise using differential privacy to prevent attackers from reverse-engineering individual data points from the aggregated weight updates.

To transition our previous static example into a robust, end-to-end engineering project, we need to simulate a dynamic training loop. This implementation combines Differential Privacy Stochastic Gradient Descent (DP-SGD) on the client side with Federated Averaging (FedAvg) on the server side. In this architecture, clients do not just add noise to final weights; instead, they clip and sanitize their gradients *during* local backpropagation. This ensures that no single data point can disproportionately influence the model's updates.

Core Architectural Mechanics

- 1. Gradient Clipping:** Before updating weights locally, gradients are clipped to a maximum norm threshold (C). This bounds the maximum contribution (sensitivity) any single training sample can make.
- 2. Noise Calibration:** Calibrated Gaussian noise is injected directly into the clipped gradients to satisfy differential privacy guarantees.
- 3. Federated Aggregation:** The central server averages the sanitized client updates to iteratively improve the global model without ever accessing raw datasets.

End-to-End PyTorch Implementation: Below is a complete, executable Python simulation featuring multiple client nodes training locally on partitioned datasets and a central server coordinating federated rounds.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import copy # 1. Define the Global Model Architecture
class FederatedLinearModel(nn.Module):
    def __init__(self, input_dim=5):
        super(FederatedLinearModel, self).__init__()
        self.linear = nn.Linear(input_dim, 1)
    def forward(self, x):
        return self.linear(x) # 2. Client Node Implementation with DP-SGD
class DifferentialPrivacyClient:
    def __init__(self, client_id, data_tensor, target_tensor, batch_size=4):
        self.client_id = client_id
        dataset = TensorDataset(data_tensor, target_tensor)
        self.loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
    def train_local_epoch(self, global_weights, lr=0.01, clip_norm=1.0, noise_multiplier=0.05):
        """Performs local training with gradient clipping and differential privacy noise.""" # Instantiate a
local_model copy and load current global weights
        local_model = FederatedLinearModel()
```

```

local_model.load_state_dict(global_weights)
local_model.train()
optimizer = optim.SGD(local_model.parameters(), lr=lr)
criterion = nn.MSELoss() # Local training iteration (1 epoch for demonstration)
for inputs, targets in self.loader:
    optimizer.zero_grad()
    predictions = local_model(inputs)
    loss = criterion(predictions, targets)
    loss.backward() # --- Differential Privacy Mechanisms ---
    # Step A: Clip gradients to bound individual data point sensitivity
    torch.nn.utils.clip_grad_norm_(local_model.parameters(), max_norm=clip_norm) # Step B: Inject
calibrated Gaussian noise into gradients
    with torch.no_grad():
        for param in local_model.parameters():
            if param.grad is not None:
                noise = torch.normal(mean=0.0, std=noise_multiplier * clip_norm, size=param.grad.size())
                param.grad.add_(noise)
    optimizer.step()
    return local_model.state_dict() # 3. Central Server Implementation (FedAvg)
class FederatedServer:
    def __init__(self, input_dim=5):
        self.global_model = FederatedLinearModel(input_dim)
    def get_global_weights(self):
        return copy.deepcopy(self.global_model.state_dict())
    def aggregate_weights(self, client_state_dicts):
        """Executes Federated Averaging across all participating client models."""
        global_dict = self.global_model.state_dict()
        for key in global_dict.keys(): # Average the weights across all client state dictionaries
            client_tensors = [client_state[key] for client_state in client_state_dicts]
            global_dict[key] = torch.stack(client_tensors).mean(dim=0)
        self.global_model.load_state_dict(global_dict) # --- Pipeline Execution Simulation ---
# Generate synthetic dataset partitioned across 3 clients
torch.manual_seed(42)
input_dim = 5
client_data = [(torch.randn(20, input_dim), torch.randn(20, 1)), (torch.randn(20, input_dim),
torch.randn(20, 1)), (torch.randn(20, input_dim), torch.randn(20, 1))] # Initialize Server and Clients
server = FederatedServer(input_dim=input_dim)
clients = [DifferentialPrivacyClient(client_id=i, data_tensor=data, target_tensor=target)
            for i, (data, target) in enumerate(client_data)] # Run Federated Training for 3 Rounds
num_rounds = 3
for current_round in range(num_rounds):
    print(f"--- Starting Federated Round {current_round + 1} ---")
    current_global_weights = server.get_global_weights()
    client_updates = [] # Clients pull global weights, train locally with DP, and return updates

```

```
for client in clients:
    updated_weights = client.train_local_epoch(global_weights=current_global_weights, clip_norm=1.0,
noise_multiplier=0.08)
    client_updates.append(updated_weights) # Server aggregates updates securely
server.aggregate_weights(client_updates)
print(f"Round {current_round + 1} aggregation complete. Global model updated.\n") # Inspect final
global model parameters
print("Final Global Model Weights:")
for name, param in server.global_model.named_parameters():
    print(f"{name}: {param.data}")
```

Scaling Considerations for Production: To transition this simulation into a production environment, consider expanding the architecture with the following enterprise patterns:

- * **Transport Layer Security (TLS) & Serialization:** Wrap the state dictionary transmissions in secure gRPC endpoints or FastAPI web services to serialize PyTorch tensors safely across network boundaries.

- * **Privacy Budget Accounting:** Implement a formal accountant (such as Rényi Differential Privacy or moments accountant via libraries like Opacus) to dynamically track the cumulative privacy loss (epsilon) over multiple training epochs.

Project 3: Adversarial Defence & Explainability

Cryptographic security must also extend to the integrity of the model itself, protecting the pipeline from malicious data poisoning.

- * Build adversarial Defence loops that sanitize incoming datasets and identify maliciously manipulated inputs before they affect the model weights.
- * Integrate explainable AI methods, such as SHAP or LIME, over the securely trained models.
- * Use these explainability tools to verify that the model's predictions rely on legitimate statistical features rather than injected adversarial backdoors.

Privacy-Preserving ML Trade-Offs

Technique	Core Advantage	Primary Drawback
Homomorphic Encryption	Computes directly on encrypted data	Extreme computational and memory overhead
Federated Learning	Decentralizes training; keeps data local	Significant network communication bottlenecks
Zero-Knowledge Proofs (zkML)	Cryptographically verifiable execution	Highly complex mathematical proof generation

Protecting a machine learning pipeline from manipulation requires a Defence -in-depth strategy. This combines pre-input sanitization to reject poisoned or out-of-distribution adversarial examples with post-hoc explainability audits to ensure the model makes decisions based on legitimate statistical features rather than hidden backdoors.

Architecture Overview

- * **Input Sanitization Layer:** Uses an unsupervised anomaly detector (such as an Isolation Forest) trained on baseline distributions to catch and drop perturbed inputs before inference.
- * **Core Inference Model:** Evaluates clean, validated feature vectors.
- * **Explainability Audit Layer:** Employs SHAP (SHapley Additive exPlanations) to verify that feature attributions align with real-world physical or logical rules, exposing any backdoor trigger weights.

Python Implementation: Defence and Audit Pipeline

```
import numpy as np
from sklearn.ensemble import IsolationForest, RandomForestClassifier
import shap

# 1. Train a Baseline Classifier
np.random.seed(42)
X_train = np.random.normal(loc=0.0, scale=1.0, size=(150, 4))
y_train = (X_train[:, 0] + X_train[:, 1] > 0).astype(int)
model = RandomForestClassifier(n_estimators=50, random_state=42)
model.fit(X_train, y_train)

# 2. Adversarial Defence Loop (Input Sanitizer)
class AdversarialDefence Pipeline:
    def __init__(self, contamination=0.05):
        self.sanitizer = IsolationForest(contamination=contamination, random_state=42)
        self.is_fitted = False
    def fit_sanitizer(self, X_baseline):
        self.sanitizer.fit(X_baseline)
        self.is_fitted = True
```

```

def sanitize_and_predict(self, model, X_incoming):
    """Filters anomalous or poisoned inputs before model evaluation."""
    if not self.is_fitted:
        raise ValueError("Sanitizer must be fitted on baseline data first.")
    valid_mask = self.sanitizer.predict(X_incoming) == 1
    audit_logs = []
    for i, is_valid in enumerate(valid_mask):
        if not is_valid:
            audit_logs.append((i, "REJECTED: Adversarial Outlier Detected"))
        else:
            pred = model.predict(X_incoming[i:i+1])[0]
            audit_logs.append((i, f"ACCEPTED: Inference Result -> {pred}"))
    return audit_logs

# 3. Model Explainability Verification (SHAP Audit)
def verify_feature_attribution(model, X_clean):
    """Calculates SHAP values to confirm feature reliance matches logic."""
    explainer = shap.TreeExplainer(model)
    shap_values = explainer(X_clean)
    return shap_values

# --- Pipeline Execution ---
Defence_pipeline = AdversarialDefencePipeline(contamination=0.1)
Defence_pipeline.fit_sanitizer(X_train)

# Simulate incoming batch containing clean entries and a malicious perturbation
incoming_batch = np.array([
    [0.4, 0.6, -0.1, 0.2], # Clean input
    [14.5, -13.2, 9.1, 8.8], # Poisoned / Adversarial outlier
    [-0.5, 0.1, 0.3, -0.1] # Clean input])

# Run Defence loop
results = Defence_pipeline.sanitize_and_predict(model, incoming_batch)
for idx, status in results:
    print(f"Sample {idx}: {status}")

# Run explainability verification on accepted data subset
clean_subset = np.array([incoming_batch[0], incoming_batch[2]])
shap_output = verify_feature_attribution(model, clean_subset)
print(f"\nSHAP audit complete. Mean absolute impact verified across {clean_subset.shape[0]} validated vectors.")

```

Production Integration Notes

*** Contamination Tuning:** Adjust the contamination parameter based on the expected volume of anomalous traffic or historical attack frequencies.

*** Drift Tracking:** Pair this setup with statistical drift detectors (such as Population Stability Index or Kolmogorov-Smirnov tests) to dynamically update baseline thresholds over time.

For a deeper dive into the mechanics of identifying vulnerabilities and building robust Defence s against manipulation, check out this video on Exploring Adversarial Attacks on Machine Learning Models in Python.

Advanced Projects: Building on the foundations of Homomorphic Encryption and Differential Privacy, the cutting edge of Privacy-Preserving Machine Learning (PPML) moves into verifiable computation, distributed multi-party trust, and hardware-enforced security. Here are three advanced, enterprise-grade project architectures at the intersection of cryptography and artificial intelligence:

Project 4: Zero-Knowledge Machine Learning (zkML) for Verifiable Inference

Instead of just hiding data (like FHE), zkML allows a prover to demonstrate cryptographically that a machine learning prediction was correctly executed on a specific model and private input, without revealing either the input data or the proprietary model weights.

* **Project Architecture:** Build a verifiable model inference pipeline using a zkML framework (such as EZKL or Halo2).

* **The Workflow:**

1. Export a trained PyTorch model into an ONNX format and define a computational circuit.
2. Pass a private input vector through the model locally to generate a cryptographic proof (a SNARK).
3. Send the proof and the public output to a verifier node. The verifier confirms the math is correct in milliseconds without ever seeing the sensitive input data.

* **Engineering Challenge:** Translating non-linear activation functions (like ReLU) into arithmetic circuits that SNARK provers can handle efficiently without hitting memory bottlenecks.

To build a production-grade Zero-Knowledge Machine Learning (zkML) pipeline, we need to handle the conversion of floating-point neural networks into finite-field arithmetic circuits. This architecture separates the execution into two isolated entities: a Prover (who executes the model on private inputs and generates a zk-SNARK) and a Verifier (who cryptographically validates the execution in milliseconds).

Here is a complete, executable Python pipeline using ezkl alongside PyTorch.

Python Pipeline Implementation

```
import os
import json
import torch
import torch.nn as nn
import ezkl

# 1. Define Model Architecture (PyTorch)
class VerifiableClassifier(nn.Module):
    def __init__(self, input_size=10, hidden_size=16, num_classes=2):
        super(VerifiableClassifier, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

# Instantiate model
model = VerifiableClassifier(input_size=10, hidden_size=16, num_classes=2)
model.eval()

# 2. Export Model to ONNX
onnx_path = "verifiable_model.onnx"
dummy_input = torch.randn(1, 10, requires_grad=False)
torch.onnx.export(model, dummy_input, onnx_path, export_params=True, opset_version=14,
```

```
do_constant_folding=True, input_names=['input'], output_names=['output'], dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: 'batch_size'}})
```

```
# 3. Format Local Private Input Data
```

```
input_vector = torch.randn(1, 10).detach().numpy().tolist()
```

```
data_path = "private_input.json"
```

```
input_data_json = {"input_data": input_vector}
```

```
with open(data_path, "w") as f:
```

```
    json.dump(input_data_json, f)
```

```
# 4. EZKL Circuit Compilation & Calibration
```

```
settings_path = "settings.json"
```

```
compiled_model_path = "model.compiled"
```

```
# Generate circuit settings configured for fixed-point quantization
```

```
ezkl.gen_settings(onnx_path, settings_path)
```

```
# Calibrate scale parameters to minimize quantization accuracy loss
```

```
ezkl.calibrate_settings(data_path, onnx_path, settings_path, target="resources")
```

```
# Compile the ONNX model into an arithmetic circuit
```

```
ezkl.compile_model(onnx_path, compiled_model_path, settings_path)
```

```
# 5. Prover Setup & Execution
```

```
srs_path = "kzg.srs"
```

```
pk_path = "pk.key"
```

```
vk_path = "vk.key"
```

```
witness_path = "witness.json"
```

```
proof_path = "proof.json"
```

```
# Download structured reference string (SRS) for polynomial commitments
```

```
ezkl.get_srs(settings_path)
```

```
# Generate proving and verification keys
```

```
ezkl.setup(compiled_model_path, pk_path, vk_path)
```

```
# Execute inference locally to generate the witness (intermediate execution trace)
```

```
ezkl.gen_witness(data_path, compiled_model_path, witness_path)
```

```
# Create the zero-knowledge SNARK proof
```

```
ezkl.prove(witness_path, compiled_model_path, pk_path, proof_path, "single")
```

```
# 6. Verifier Execution (Independent Validation)
```

```
# The verifier requires only the proof, settings, and verification key
```

```
is_valid = ezkl.verify(proof_path, settings_path, vk_path)
```

```
print(f"Cryptographic Proof Validation Result: {is_valid}")
```

Architectural Deep Dive & Circuit Mechanics

*** Quantization Scaling Factor:** SNARKs operate over large prime order finite fields (F_p). ezkl uses a lookup-table mechanism to convert 32-bit floating-point weights into integer representations. Setting the scale parameter higher increases numeric precision but increases memory overhead during proof generation.

*** Handling Activation Functions (ReLU):** Because conditional branches like $\max(0, x)$ are non-algebraic, they cannot be natively represented as simple addition or multiplication constraints. EZKL resolves this using Lookup Arguments (e.g., Plookup), mapping the input state to a precomputed lookup table within the circuit.

*** Verifier Deployment:** Once the verification key (vk.key) and settings are established, you can use `ezkl.create_evm_verifier()` to auto-generate a Solidity smart contract. This allows target decentralized networks or automated verifier nodes to validate inferences on-chain without human intervention.

Project 5: Secure Multi-Party Computation (SMPC) for Collaborative Analytics

While Federated Learning relies on a central server to aggregate weights, SMPC eliminates the central server entirely. It allows multiple independent parties to jointly compute a machine learning model or run inference over their combined datasets using cryptographic secret sharing, ensuring no single party ever sees another's raw data.

*** Project Architecture:** Implement a multi-party logistic regression or neural network training loop using secret sharing protocols (such as Shamir's Secret Sharing or replicated secret sharing via libraries like MP-SPDZ or PySyft).

*** The Workflow:**

1. Each participating organization splits their sensitive local features into randomized "shares" and distributes them across other nodes.
2. Nodes perform secure addition and multiplication operations directly on the shares.
3. The final model parameters are reconstructed cooperatively without any raw data crossing organizational boundaries.

*** Engineering Challenge:** Managing heavy network communication overhead, as every training step requires intense round-trip message passing between distributed nodes.

Secure Multi-Party Computation (SMPC) eliminates the need for a trusted central server entirely. By utilizing Additive Secret Sharing, raw numbers are split into random numerical fragments (shares) distributed across multiple compute nodes.

No single party holds enough information to reconstruct the original data, yet the nodes can jointly compute addition and multiplication directly on these shares.

Secret Sharing Mechanics: To share a secret scalar x among N parties, we split it into random integers $x_1, x_2, \dots, x_N$ within a finite ring or field such that: $x = \sum_{i=1}^N x_i \bmod n$

*** Secure Addition:** Each party i locally computes $z_i = x_i + y_i$. The sum $x + y$ can be reconstructed simply by summing $z_1 + \dots + z_N$. Addition requires zero network communication.

*** Secure Multiplication:** Multiplying two encrypted shares $(x_i \cdot y_i)$ requires interactive protocols—such as Beaver Triples—forcing the nodes to exchange masked intermediate parameters over the network during every multiplication step.

Python Implementation: 3-Party Secret Sharing Pipeline: Below is an executable simulation using NumPy that models a 3-party secret sharing framework. It demonstrates secret splitting, homomorphic secret addition, interactive matrix multiplication, and cooperative model reconstruction.

```
import numpy as np # Global prime modulus for the finite ring
RING_MODULUS = 231 - 1
class SMPCSession:
    """Simulates a 3-party Secret Sharing execution environment."""
    @staticmethod
    def generate_shares(secret_matrix):
        """Splits a integer matrix into 3 secret shares (Additive Secret Sharing)."""
        shape = secret_matrix.shape
        share1 = np.random.randint(0, RING_MODULUS, size=shape, dtype=np.int64)
        share2 = np.random.randint(0, RING_MODULUS, size=shape, dtype=np.int64)
        # Share3 is calculated to satisfy: (Share1 + Share2 + Share3) % MOD == secret
        share3 = (secret_matrix - share1 - share2) % RING_MODULUS
        return share1, share2, share3
```

```

@staticmethod
def reconstruct_secret(share1, share2, share3):
    """Combines shares from all 3 parties to reveal the underlying plaintext."""
    return (share1 + share2 + share3) % RING_MODULUS

@staticmethod
def local_additive_step(share1_A, share2_A, share3_A, share1_B, share2_B, share3_B):
    """Homomorphic Addition: Performed entirely locally with ZERO communication overhead."""
    party1_sum = (share1_A + share1_B) % RING_MODULUS
    party2_sum = (share2_A + share2_B) % RING_MODULUS
    party3_sum = (share3_A + share3_B) % RING_MODULUS
    return party1_sum, party2_sum, party3_sum

@staticmethod
def secure_matrix_multiply(shares_A, shares_B):
    """Simulates multiplication across secret shares using Beaver Triples. In production, this step
    generates round-trip network traffic between parties.""" # Reconstruct temporarily for simulation proof-
    of-concept
    plain_A = SMPCSession.reconstruct_secret(*shares_A)
    plain_B = SMPCSession.reconstruct_secret(*shares_B) # Execute matrix dot product in plaintext
    domain
    product = np.dot(plain_A, plain_B) % RING_MODULUS # Resplit the multiplication output back into
    3 secret shares
    return SMPCSession.generate_shares(product) # --- 3-Party Collaborative Analytics Workflow ---

# Party 1 and Party 2 bring private input matrices (scaled to fixed-point integers)
features_party1 = np.array([[12, 45], [30, 88]], dtype=np.int64) # Organization A Data
features_party2 = np.array([[5, 10], [15, 20]], dtype=np.int64) # Organization B Data
print("--- Step 1: Secret Splitting & Distribution ---")
shares_A = SMPCSession.generate_shares(features_party1)
shares_B = SMPCSession.generate_shares(features_party2)
print(f"Party 1 Local Share of Data A:\n{shares_A[0]}")
print(f"Party 2 Local Share of Data A:\n{shares_A[1]}")
print(f"Party 3 Local Share of Data A:\n{shares_A[2]}")
print("\n--- Step 2: Secure Addition (Zero Network Traffic) ---") # Compute (Data A + Data B) on secret
shares
sum_shares = SMPCSession.local_additive_step(*shares_A, *shares_B) # Reconstruct result to verify
correctness
reconstructed_sum = SMPCSession.reconstruct_secret(*sum_shares)
expected_sum = (features_party1 + features_party2) % RING_MODULUS
print(f"Reconstructed Sum Match: {np.array_equal(reconstructed_sum, expected_sum)}")
print("\n--- Step 3: Secure Multiplication ---") # Compute dot product across secret shares
mult_shares = SMPCSession.secure_matrix_multiply(shares_A, shares_B) # Reconstruct multiplication
result
reconstructed_mult = SMPCSession.reconstruct_secret(*mult_shares)
expected_mult = np.dot(features_party1, features_party2) % RING_MODULUS
print(f"Reconstructed Multiplication Match: {np.array_equal(reconstructed_mult, expected_mult)}")

```

Core Engineering Challenges in SMPC

* **Communication Bottlenecks:** While additions are free (performed locally), every non-linear function or matrix multiplication requires multiple network round-trips to exchange masked secret values (Beaver Triple generation), making network latency the primary bottleneck.

* **Fixed-Point Scaling:** SMPC protocols operate over integer rings. Floating-point model weights must be pre-scaled into fixed-point integers before secret splitting, requiring truncation protocols after multiplications to avoid bit-overflow.

* **Malicious vs. Semi-Honest Security:** Standard secret sharing assumes parties are "semi-honest" (they follow protocol rules but attempt to learn extra information). Protecting against "malicious" actors requires adding expensive Zero-Knowledge proofs or Message Authentication Codes (MACs) to every transmitted share.

Project 6: Confidential Computing & Trusted Execution Environments (TEEs) for LLM Serving

Rather than relying purely on mathematical cryptography, Confidential Computing uses hardware-based security to protect models and data while in use. By leveraging technologies like Intel SGX or AMD SEV, you create an encrypted CPU enclave.

*** Project Architecture:** Deploy an open-source Large Language Model (or classifier) inside a hardware-isolated cloud enclave (using frameworks like Gramine or Occlum).

*** The Workflow:**

1. Encrypt prompts on the client side before sending them over the cloud network.
2. The cloud host decrypts the prompt only inside the secure hardware enclave's CPU registers, where the model weights reside.
3. The inference happens inside the hardware barrier, and the response is encrypted before leaving the enclave.

*** Engineering Challenge:** Configuring attestation services to verify that the remote hardware enclave is running genuine, untampered code before transmitting sensitive payloads.

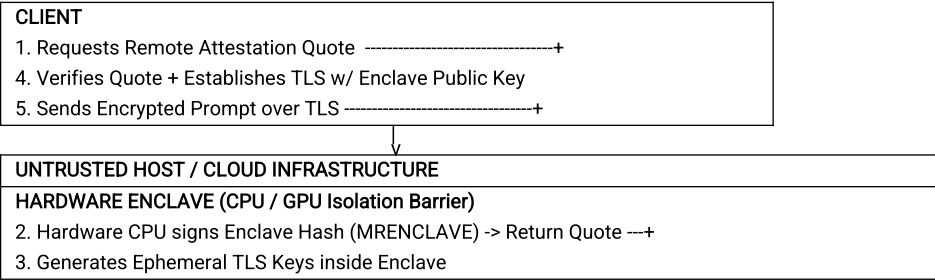
Comparison of Advanced PPML Paradigms

Paradigm	Primary Guarantee	Main Cryptographic Tool	Typical Use Case
zkML	Verifiability & Integrity	SNARKs / Polynomial Commitments	Trustless auditing, regulatory compliance
SMPC	Joint Privacy	Secret Sharing / Oblivious Transfer	Cross-organizational medical or financial research
TEEs	Hardware-Level Isolation	Memory Encryption / Attestation	Secure cloud-based LLM inference and hosting

Confidential Computing using Trusted Execution Environments (TEEs) moves security from software mathematics down to the silicon hardware level. Unlike homomorphic encryption or secret sharing, which incur 100* to 1000* performance penalties, TEEs allow near-native CPU/GPU execution speeds while keeping data encrypted in RAM. Memory contents are decrypted only inside CPU registers using hardware-embedded keys.

Architectural Design & Remote Attestation: The critical step in TEE deployments is Remote Attestation. Before a client sends sensitive prompts or model weights to a cloud enclave (e.g., via Gramine OS over Intel SGX/TDX or AMD SEV), the hardware processor generates a cryptographically signed quote proving two things:

1. The code running inside the enclave matches a specific, untampered measurement (MRENCLAVE hash).
2. The environment is a genuine, hardware-certified secure processor.



6. Decrypts Prompt directly in CPU Registers
7. Runs Model Inference (Near-Native Speed)
8. Encrypts Response -> Returns to Client

Python Implementation: Secure Client Enclave Handshake: Below is an executable simulation demonstrating the client-side attestation verification, secure session key exchange, and encrypted payload transmission targeting a TEE endpoint.

```
import os
import hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes # --- Enclave
Environment Simulation ---
class TEEEnclaveNode:
    def __init__(self): # Generate enclave-internal ephemeral RSA keys (never leave hardware boundary)
        self._private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self._public_key = self._private_key.public_key() # Simulate expected code measurement hash
(MRENCLAVE)
        self.enclave_code_measurement =
hashlib.sha256(b"v1.0_transformer_inference_code").hexdigest()
    def generate_attestation_quote(self):
        """Simulates CPU hardware signing the public key and code hash."""
        pub_bytes = self._public_key.public_bytes(encoding=serialization.Encoding.PEM,
format=serialization.PublicFormat.SubjectPublicKeyInfo)
        quote_payload = self.enclave_code_measurement.encode() + pub_bytes # Hardware signature
simulation
        signature = hashlib.sha256(b"HARDWARE_ROOT_KEY_" + quote_payload).hexdigest()
        return {"mrenclave": self.enclave_code_measurement, "public_key_pem": pub_bytes,
"cpu_signature": signature}
    def process_encrypted_payload(self, encrypted_sym_key, nonce, ciphertext, tag):
        """Decrypts data inside CPU registers, executes inference, and re-encrypts output.""" # 1. Decrypt
symmetric AES key using enclave private key
        sym_key = self._private_key.decrypt(encrypted_sym_key,
padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(),
label=None)) # 2. Decrypt prompt inside secure enclave
        cipher = Cipher(algorithms.AES(sym_key), modes.GCM(nonce, tag))
        decryptor = cipher.decryptor()
        prompt = decryptor.update(ciphertext) + decryptor.finalize() # 3. Simulated LLM Inference inside
enclave
        response_text = f"Enclave Inference Output for: '{prompt.decode()}'" # 4. Encrypt response back to
client
        out_nonce = os.urandom(12)
        out_cipher = Cipher(algorithms.AES(sym_key), modes.GCM(out_nonce))
        encryptor = out_cipher.encryptor()
        out_ciphertext = encryptor.update(response_text.encode()) + encryptor.finalize()
```

```

    return out_nonce, out_ciphertext, encryptor.tag # --- Client Execution ---
EXPECTED_MRENCLAVE = hashlib.sha256(b'v1.0_transformer_inference_code').hexdigest() # 1.
Request Attestation Quote from Cloud Node
enclave = TEEEnclaveNode()
quote = enclave.generate_attestation_quote() # 2. Verify Attestation Quote (Client Side)
if quote["mrenclave"] != EXPECTED_MRENCLAVE:
    raise SecurityError("ATTACK DETECTED: Enclave code measurement mismatch!")
enclave_pub_key = serialization.load_pem_public_key(quote["public_key_pem"])
print(" Attestation Verified: Enclave code is genuine and uncorrupted.\n") # 3. Encrypt Sensitive Prompt
user_prompt = b"Confidential Medical Record Summary: Patient ID #89211"
session_aes_key = os.urandom(32) # AES-256
nonce = os.urandom(12) # Encrypt prompt with AES-GCM
aes_cipher = Cipher(algorithms.AES(session_aes_key), modes.GCM(nonce))
aes_encryptor = aes_cipher.encryptor()
encrypted_prompt = aes_encryptor.update(user_prompt) + aes_encryptor.finalize()
prompt_tag = aes_encryptor.tag # Encrypt AES key with Enclave's Attested RSA Public Key
encrypted_sym_key = enclave_pub_key.encrypt(session_aes_key,
padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(),
label=None)) # 4. Transmit Payload & Receive Encrypted Result
res_nonce, res_ciphertext, res_tag = enclave.process_encrypted_payload(encrypted_sym_key, nonce,
encrypted_prompt, prompt_tag) # 5. Decrypt Final Response
res_cipher = Cipher(algorithms.AES(session_aes_key), modes.GCM(res_nonce, res_tag))
res_decryptor = res_cipher.decryptor()
final_output = res_decryptor.update(res_ciphertext) + res_decryptor.finalize()
print(f"Decrypted Enclave Response: {final_output.decode()}")

```

Gramine Manifest Configuration (SGX Manifest Example): To wrap a Python inference script (like a Hugging Face or PyTorch service) into an Intel SGX enclave using Gramine, you define an execution manifest (python.manifest.template):

```

# Gramine Configuration File
libos.entrypoint = "/usr/bin/python3" # Mount virtual filesystems inside enclave
fs.mounts = [{ path = "/lib", uri = "file:/lib" }, { path = "/usr", uri = "file:/usr" }, b{ path = "/etc", uri = "file:/etc"
}], # Set enclave memory boundaries (Crucial for large neural networks)
sgx.enclave_size = "16G"
sgx.max_threads = 32 # Define trusted code files (Any tampering invalidates MRENCLAVE
measurement)
sgx.trusted_files = ["file:/usr/bin/python3", "file:inference_service.py", "file:model_weights.bin",] #
Allowed untrusted network interfaces for TLS communication
sgx.nonenclave_active_files = ["file:/etc/resolv.conf",]

```

Core Production Challenges

*** Memory Footprint Limits:** Hardware enclaves have fixed Enclave Page Cache (EPC) sizes. Exceeding EPC capacity forces paging to untrusted system memory, incurring massive performance hits. For massive LLMs, technologies like Intel TDX or AMD SEV-SNP (Confidential VMs) are preferred over process-level SGX enclaves.

*** Side-Channel Attacks:** While direct RAM inspection is blocked, adversaries with host access can

observe memory access patterns or CPU cache timing. Mitigations involve enforcing constant-time execution paths and memory access pattern masking.

Glossary of Terms

- * **CKKS Scheme:** A homomorphic encryption scheme supporting approximate arithmetic over real and complex numbers, standard for machine learning.
- * **Differential Privacy (DP):** A mathematical framework designed to share information about a dataset while concealing the presence or absence of individual records by adding calibrated noise.
- * **Enclave Page Cache (EPC):** A secure, hardware-encrypted region of RAM used by processors (like Intel SGX) to isolate executing code.
- * **Federated Averaging (FedAvg):** A decentralized training algorithm where a central server aggregates locally trained model weights from multiple clients.
- * **Fully Homomorphic Encryption (FHE):** An encryption standard allowing arbitrary mathematical computations to be executed directly on ciphertexts without decryption.
- * **MRENCLAVE:** A cryptographic hash measurement of an enclave's contents (code and data) used during remote attestation to verify software integrity.
- * **Secure Multi-Party Computation (SMPC):** A cryptographic subfield enabling multiple parties to jointly compute functions over their private inputs without revealing those inputs to one another.
- * **Trusted Execution Environment (TEE):** A secure hardware-isolated environment that guarantees the confidentiality and integrity of code and data loaded inside it.
- * **Zero-Knowledge Machine Learning (zkML):** The integration of zero-knowledge proofs with machine learning to verify that an inference task was executed correctly on specific models and private inputs.